ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп’ютерних технологій

ЗВІТ

про виконання лабораторної роботи №3

«Ітератори, генератори та потокова обробка даних»

з дисципліни «Професійний Python»

Виконав: студент групи ФеП-33с
Цястун Павло
Варіант 1
Перевірив: професор Свелеба Сергій Андрійович

Львів – 2026

1. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, yield, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних.

2. Номер варіанта та постановка задачі

Варіант 1. Потокова обробка даних системи обліку студентів.

Необхідно розробити потоковий конвеєр обробки великого CSV-файла із записами студентів, у якому дані послідовно проходять через декілька lazy stages без повної матеріалізації проміжних колекцій. Реалізовано:

- потокове читання CSV та валідацію записів студентів;
- фільтрацію за групою та мінімальним середнім балом;
- lazy-пошук студентів за прізвищем або ID;
- streaming average середнього бала;
- визначення min/max average_grade без повної матеріалізації даних;
- перші N студентів із найвищим балом (heapq.nlargest, у пам’яті лише N записів) та перші N записів за умовою (islice);
- batch processing записів студентів;
- підрахунок кількості студентів за групами (Counter).

Також виконано завдання підвищеної складності: власний iterator class, recursive generator із yield from, infinite generator + islice, itertools.chain, accumulate, pairwise, groupby, dynamic batching, pipeline із декількох файлів, streaming export у новий файл, підрахунок invalid records, lazy search із early termination, вимірювання time-to-first-result та порівняння пам’яті list проти generator.

3. Опис input dataset

Формат вхідного CSV-файла (варіант 1):

```
student_id,first_name,last_name,group,average_grade
```

Для перевірки логіки використано невеликий файл data/input.csv (16 рядків). Він містить і коректні записи, і некоректні: нечислову оцінку (error), оцінку поза діапазоном 0–100 (102), порожнє ім’я та нечисловий

ідентифікатор. Некоректні рядки пропускаються й підраховуються (12 коректних, 4 некоректних).

```
student_id,first_name,last_name,group,average_grade
1,Pavlo,Tsiastun,FeP-33s,93.4
2,Olena,Savchuk,FeP-33s,87.1
3,Taras,Hrytsenko,FeP-31,71.8
4,Sofiia,Lysenko,FeP-32,96.0
5,Maksym,Kravets,FeP-31,64.5
6,Kateryna,Moroz,FeP-33s,78.9
7,Dmytro,Bilyk,FeP-32,82.3
8,Yulia,Pavlenko,FeP-31,90.2
9,Bohdan,Rudenko,FeP-32,55.7
10,Iryna,Kovalenko,FeP-33s,85.6
11,Invalid,Student,FeP-34,error
12,Petro,Novak,FeP-34,102
13,,Empty,FeP-34,70
x,Wrong,Id,FeP-31,80
14,Andrii,Shevchuk,FeP-34,88.8
15,Mariia,Boyko,FeP-34,79.3
```

Для експерименту модуль generate_data.py створює великі файли з 10 000, 100 000, 500 000 та 1 000 000 записів (випадкові групи FeP-31, FeP-32, FeP-33s, FeP-34 та оцінки від 40 до 100). Ці файли розміщуються в data/generated/ і не додаються до Git.

4. Структура проєкту

```
stream_processor_lab/
├── pyproject.toml
├── README.md
├── .gitignore
├── data/
│   └── input.csv
├── src/
│   └── stream_processor/
│       ├── __init__.py
│       ├── main.py
│       ├── models.py
│       ├── readers.py
│       ├── parsers.py
│       ├── filters.py
│       ├── transformations.py
│       ├── batches.py
│       ├── analytics.py
│       ├── pipeline.py
│       ├── iterators.py
│       ├── exporter.py
│       ├── generate_data.py
│       └── benchmark.py
└── tests/
    └── test_stream.py
```

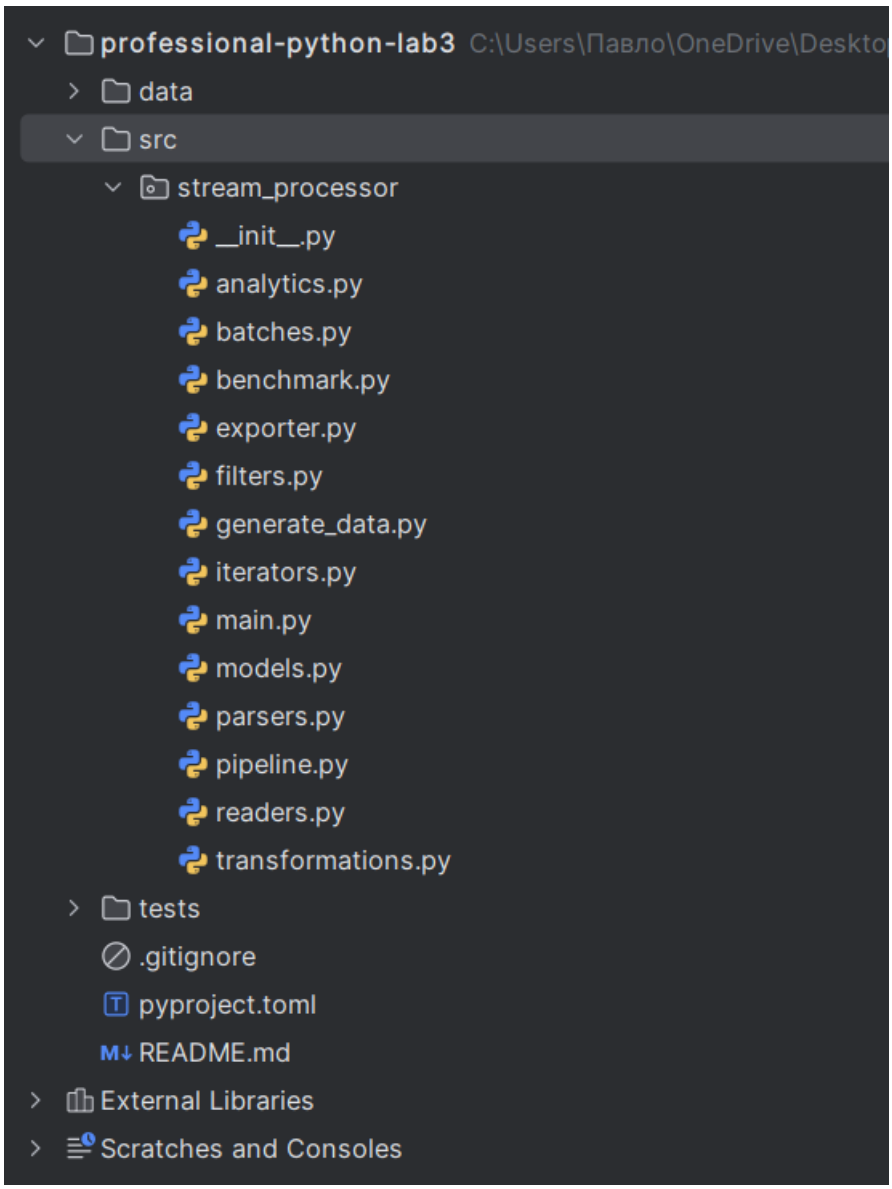


Рис. 1. Структура проєкту в PyCharm

5. Iterable та iterator

Iterable — об’єкт, з якого можна отримати iterator викликом iter() (наприклад, list, файл, власний клас із методом __iter__). Iterator — об’єкт, який зберігає поточний стан обходу, повертає наступний елемент методом __next__() і піднімає StopIteration, коли елементи закінчились. Кожен iterator також є iterable: iter(iterator) повертає той самий об’єкт.

Цикл for автоматично викликає iter(), послідовно викликає next() та завершується при StopIteration. Той самий протокол використовують sum(), min(), max(), sorted(), list(), tuple() та comprehensions.

Iterator зазвичай є one-shot object: після вичерпання повторний обхід повертає порожній результат. Тому конвеєр у програмі створюється заново для кожного споживача.

6. Власний iterator, generator function та yield

6.1. Власний iterator class

Контейнер NumberRange (iterable) відокремлено від NumberRangeIterator (iterator). Завдяки цьому кожен виклик iter() створює новий незалежний iterator, і NumberRange можна обходити декілька разів.

```
"""Iterator protocol examples: classes, infinite and recursive generators."""

from collections.abc import Iterable, Iterator
from itertools import count, islice, takewhile



class NumberRange:
    """Iterable: every iter() call creates a new independent iterator."""

    def __init__(self, start: int, stop: int) -> None:
        self.start = start
        self.stop = stop

    def __iter__(self) -> "NumberRangeIterator":
        return NumberRangeIterator(self.start, self.stop)



class NumberRangeIterator:
    """Iterator: keeps the state of traversal and implements __next__."""

    def __init__(self, start: int, stop: int) -> None:
        self.current = start
        self.stop = stop

    def __iter__(self) -> "NumberRangeIterator":
        return self

    def __next__(self) -> int:
        if self.current >= self.stop:
            raise StopIteration

        value = self.current
        self.current += 1
        return value



def natural_numbers(start: int = 1) -> Iterator[int]:
    """Infinite generator."""
    value = start
    while True:
        yield value
        value += 1
```

```
def first_squares(n: int) -> list[int]:
    """Infinite generator limited with islice."""
    return list(islice((x * x for x in natural_numbers()), n))


def squares_below(limit: int) -> list[int]:
    """Infinite stream limited with takewhile."""
    return list(takewhile(lambda x: x < limit, (x * x for x in count(1))))


def flatten(values: Iterable) -> Iterator:
    """Recursive generator with yield from."""
    for value in values:
        if isinstance(value, list):
            yield from flatten(value)
        else:
            yield value
```

6.2. Generator functions і yield

Функція, яка містить yield, є generator function. Її виклик не виконує тіло, а створює generator object. Під час кожного next() виконання продовжується до наступного yield, після чого функція призупиняється, зберігаючи локальні змінні та позицію. У проєкті generator functions — це read_lines, read_many, parse_csv_rows, validate_students, filter_by_group, filter_by_grade, normalize_names, batched, dynamic_batches, natural_numbers і flatten.

6.3. yield from

Конструкцію yield from використано для делегування генерації: у readers.read_lines (рядки файла), parsers.parse_csv_rows (рядки csv.DictReader), read_many (рядки наступних файлів) та в рекурсивному flatten.

6.4. Generator expression

```
# analytics.running_totals / grade_differences
grades = (s.average_grade for s in students)

# transformations.to_short_view
(f"#{s.student_id} {s.full_name}: {s.average_grade:.1f}" for s in students)

# filters.find_by_last_name
(s for s in students if s.last_name.lower() == needle)
```

Generator expression не створює проміжний list і обчислює елементи лише за потреби.

7. Використані засоби itertools

- islice — перші N записів потоку (first_n), batching (batched), обмеження нескінченного генератора (first_squares);
- chain — об’єднання двох pipelines у єдиний потік;
- accumulate — накопичувальна сума оцінок (running_totals);
- pairwise — різниці між сусідніми оцінками (grade_differences);
- groupby — підрахунок студентів за групами після попереднього сортування за тим самим ключем;
- count та takewhile — нескінченна послідовність, обмежена умовою (squares_below).

```
"""Aggregation stage: streaming statistics."""

import heapq
from collections import Counter
from collections.abc import Iterable, Iterator
from itertools import accumulate, groupby, islice, pairwise

from stream_processor.models import StudentRecord



def streaming_average(students: Iterable[StudentRecord]) -> float:
    """Average grade in one pass, memory does not depend on data size."""
    total = 0.0
    count = 0

    for student in students:
        total += student.average_grade
        count += 1

    return total / count if count else 0.0



def calculate_statistics(students: Iterable[StudentRecord]) -> dict:
    """Count, average, min, max and groups in a single pass."""
    count = 0
    total = 0.0
    minimum: float | None = None
    maximum: float | None = None
    groups: Counter = Counter()

    for student in students:
        grade = student.average_grade
        count += 1
        total += grade
        groups[student.group] += 1

        if minimum is None or grade < minimum:
            minimum = grade

        if maximum is None or grade > maximum:
            maximum = grade
```

```
    return {
        "count": count,
        "average": total / count if count else 0.0,
        "minimum": minimum,
        "maximum": maximum,
        "groups": dict(groups),
    }


def top_n(students: Iterable[StudentRecord], n: int) -> list[StudentRecord]:
    """N students with the highest grade; only N records are kept in memory."""
    return heapq.nlargest(n, students, key=lambda s: s.average_grade)


def first_n(students: Iterable[StudentRecord], n: int) -> list[StudentRecord]:
    """First N records of a stream; the rest is never processed (islice)."""
    return list(islice(students, n))


def running_totals(students: Iterable[StudentRecord]) -> Iterator[float]:
    """Cumulative sum of grades (itertools.accumulate)."""
    return accumulate(s.average_grade for s in students)


def grade_differences(students: Iterable[StudentRecord]) -> Iterator[float]:
    """Differences between neighbouring grades (itertools.pairwise)."""
    grades = (s.average_grade for s in students)
    return (round(b - a, 2) for a, b in pairwise(grades))


def count_by_group_sorted(students: list[StudentRecord]) -> dict[str, int]:
    """groupby works on consecutive keys, so the data is sorted first."""
    ordered = sorted(students, key=lambda s: s.group)
    return {
        group: sum(1 for _ in members)
        for group, members in groupby(ordered, key=lambda s: s.group)
    }
```

```
"""Batching of a stream."""


from collections.abc import Iterable, Iterator
from itertools import islice
from typing import TypeVar

T = TypeVar("T")



def batched(items: Iterable[T], size: int) -> Iterator[list[T]]:
    """Split a stream into lists of ``size`` elements."""
    if size < 1:
        raise ValueError("Batch size must be positive.")

    iterator = iter(items)

    while batch := list(islice(iterator, size)):
        yield batch
```

```
def dynamic_batches(
    items: Iterable[float], limit: float
) -> Iterator[list[float]]:
    """Dynamic batching: a batch is closed when its sum reaches ``limit``."""
    batch: list[float] = []
    total = 0.0

    for item in items:
        batch.append(item)
        total += item

        if total >= limit:
            yield batch
            batch, total = [], 0.0

    if batch:
        yield batch
```

8. Схема pipeline

```
Input file (CSV)
      ↓
read_lines()          — streaming reader (yield from)
      ↓
parse_csv_rows()      — parser (csv.DictReader)
      ↓
validate_students()   — validation, лічильник invalid
      ↓
normalize_names()     — transformation
      ↓
filter_by_grade()     — filtering (+ filter_by_group)
      ↓
batched() / calculate_statistics() / export_students()
      ↓
Result
```

Усі етапи, окрім останнього, є generators. Файл не читається, доки споживач (sum, цикл for, islice) не запросить перший елемент.

9. Код етапів конвеєра

9.1. Streaming reader

```
"""Source stage: streaming reading of files."""

from collections.abc import Iterator
from pathlib import Path



def read_lines(path: Path) -> Iterator[str]:
    """Yield lines of a file one by one (the file is never read whole)."""
    with path.open("r", encoding="utf-8", newline="") as file:
        yield from file
```

```
def read_many(paths: list[Path]) -> Iterator[str]:
    """Stream several files one after another, header only from the first."""
    for number, path in enumerate(paths):
        lines = read_lines(path)

        if number > 0:
            next(lines, None)  # skip the header of the next files

        yield from lines
```

9.2. Parser

```
"""Parsing stage: lines of CSV text to dictionaries."""

import csv
from collections.abc import Iterable, Iterator



def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    """Parse CSV lines lazily using csv.DictReader."""
    yield from csv.DictReader(lines)
```

9.3. Validation stage та filtering stage

```
"""Validation and filtering stages."""

from collections import Counter
from collections.abc import Callable, Iterable, Iterator

from stream_processor.models import StudentRecord



def parse_student_row(row: dict[str, str]) -> StudentRecord | None:
    """Convert one CSV row to a record or return None if it is invalid."""
    try:
        student_id = int(row["student_id"])
        first_name = row["first_name"].strip()
        last_name = row["last_name"].strip()
        group = row["group"].strip()
        grade = float(row["average_grade"])
    except (ValueError, TypeError, KeyError, AttributeError):
        return None

    if not (first_name and last_name and group):
        return None

    if not 0 <= grade <= 100:
        return None

    return StudentRecord(student_id, first_name, last_name, group, grade)



def validate_students(
    rows: Iterable[dict[str, str]], stats: Counter | None = None
) -> Iterator[StudentRecord]:
```

```
    """Validation stage; counts valid/invalid rows in ``stats``."""
    for row in rows:
        record = parse_student_row(row)

        if record is None:
            if stats is not None:
                stats["invalid"] += 1
            continue

        if stats is not None:
            stats["valid"] += 1

        yield record



def filter_by_group(
    students: Iterable[StudentRecord], group: str
) -> Iterator[StudentRecord]:
    """Keep only students of the given group."""
    for student in students:
        if student.group == group:
            yield student



def filter_by_grade(
    students: Iterable[StudentRecord], minimum: float
) -> Iterator[StudentRecord]:
    """Keep only students with grade >= minimum."""
    for student in students:
        if student.average_grade >= minimum:
            yield student



def find_by_last_name(
    students: Iterable[StudentRecord], last_name: str
) -> Iterator[StudentRecord]:
    """Lazy search by last name (case-insensitive)."""
    needle = last_name.lower()
    return (s for s in students if s.last_name.lower() == needle)



def find_first(
    records: Iterable[StudentRecord], predicate: Callable[[StudentRecord], bool]
) -> StudentRecord | None:
    """Lazy search with early termination."""
    for record in records:
        if predicate(record):
            return record

    return None
```

9.4. Transformation stage

```
"""Transformation stages."""

from collections.abc import Iterable, Iterator

from stream_processor.models import StudentRecord
```

```
def normalize_names(students: Iterable[StudentRecord]) -> Iterator[StudentRecord]:
    """Normalize letter case of names."""
    for student in students:
        yield student._replace(
            first_name=student.first_name.title(),
            last_name=student.last_name.title(),
        )


def to_short_view(students: Iterable[StudentRecord]) -> Iterator[str]:
    """Lazy transformation into a short text representation."""
    return (
        f"#{s.student_id} {s.full_name} ({s.group}): {s.average_grade:.1f}"
        for s in students
    )
```

9.5. Модель запису

```
"""Data model of the pipeline."""

from typing import NamedTuple


class StudentRecord(NamedTuple):
    """One validated student record (immutable, light-weight)."""

    student_id: int
    first_name: str
    last_name: str
    group: str
    average_grade: float

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

9.6. Побудова pipeline

```
"""Pipeline: connection of lazy stages."""

from collections import Counter
from collections.abc import Iterator
from pathlib import Path

from stream_processor.filters import (
    filter_by_grade,
    filter_by_group,
    validate_students,
)
from stream_processor.models import StudentRecord
from stream_processor.parsers import parse_csv_rows
from stream_processor.readers import read_lines, read_many
from stream_processor.transformations import normalize_names
```

```
def build_pipeline(
    path: Path | list[Path],
    minimum_grade: float = 0.0,
    group: str | None = None,
    stats: Counter | None = None,
) -> Iterator[StudentRecord]:
    """Connect the stages. Nothing is read until the result is consumed.

    The returned pipeline is one-shot: after it is consumed, build a new one.
    """
    lines = read_many(path) if isinstance(path, list) else read_lines(path)
    rows = parse_csv_rows(lines)
    valid = validate_students(rows, stats)
    normalized = normalize_names(valid)
    stream = filter_by_grade(normalized, minimum_grade)

    if group is not None:
        stream = filter_by_group(stream, group)

    return stream
```

9.7. Streaming export (sink)

```
"""Sink stage: streaming export to a new file."""

import csv
from collections.abc import Iterable
from pathlib import Path

from stream_processor.models import StudentRecord



def export_students(students: Iterable[StudentRecord], path: Path) -> int:
    """Write records to CSV while they pass through the pipeline."""
    written = 0
    path.parent.mkdir(parents=True, exist_ok=True)

    with path.open("w", encoding="utf-8", newline="") as file:
        writer = csv.writer(file)
        writer.writerow(
            ["student_id", "first_name", "last_name", "group", "average_grade"]
        )

        for student in students:
            writer.writerow(student)
            written += 1

    return written
```

10. Результат роботи програми

Запуск: python -m stream_processor.main (з кореня проєкту, де лежить каталог data).

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab3> python -m stream_processor.main
== 1. Iterator protocol ==
list(NumberRange(1, 5)) twice: [1, 2, 3, 4] [1, 2, 3, 4]
first 5 squares (infinite generator + islice): [1, 4, 9, 16, 25]
squares below 50 (takewhile): [1, 4, 9, 16, 25, 36, 49]
flatten (yield from): [1, 2, 3, 4, 5, 6]

== 2. Pipeline: grade >= 85 ==
pipeline is lazy, object: generator
  1 Pavlo Tsiastun FeP-33s 93.4
  2 Olena Savchuk FeP-33s 87.1
  4 Sofiia Lysenko FeP-32 96.0

== 3. Statistics (one pass, one-shot pipeline is rebuilt) ==
  count: 6
  average: 90.18333333333332
  minimum: 85.6
  maximum: 96.0
  groups: {'FeP-33s': 3, 'FeP-32': 1, 'FeP-31': 1, 'FeP-34': 1}
  valid/invalid rows: {'valid': 12, 'invalid': 4}

== 4. Counting rows of the whole file ==
  average of all valid students: 81.13, rows: {'valid': 12, 'invalid': 4}
```

Рис. 2. Результат запуску (початок виводу)

```
== 5. Search and top N ==
  by last name 'moroz': [6]
  top 3 by grade: [('Sofiia Lysenko', 96.0), ('Pavlo Tsiastun', 93.4), ('Yulia Pavlenko', 90.2)]

== 6. Batches of 4 students ==
  batch 1: ids [1, 2, 3, 4]
  batch 2: ids [5, 6, 7, 8]
  batch 3: ids [9, 10, 14, 15]

== 7. itertools ==
  accumulate: [93.4, 180.5, 276.5, 366.7, 452.3, 541.1]
  pairwise differences: [-6.3, 8.9, -5.8, -4.6, 3.2]
  groupby (after sorting): {'FeP-31': 3, 'FeP-32': 3, 'FeP-33s': 4, 'FeP-34': 2}
  dynamic batches (sum >= 200): [[90, 85, 70], [95, 60, 50], [99]]
  chain of two pipelines: [1, 4, 8, 4]

== 8. Lazy transformation ==
  #1 Pavlo Tsiastun (FeP-33s): 93.4
  #4 Sofiia Lysenko (FeP-32): 96.0
  #8 Yulia Pavlenko (FeP-31): 90.2

== 9. Streaming export ==
  6 records written to data\generated\export_85.csv
```

Рис. 3. Результат запуску (продовження виводу)

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab3> python -m unittest discover -s tests -v
test_batched (test_stream.BatchTest.test_batched) ... ok
test_dynamic_batches (test_stream.BatchTest.test_dynamic_batches) ... ok
test_lazy_exception (test_stream.BatchTest.test_lazy_exception) ... ok
test_multi_file_pipeline (test_stream.FileTest.test_multi_file_pipeline) ... ok
test_exhausted_generator (test_stream.IteratorTest.test_exhausted_generator) ... ok
test_flatten (test_stream.IteratorTest.test_flatten) ... ok
test_infinite_generator_with_islice (test_stream.IteratorTest.test_infinite_generator_with_islice) ... ok
test_range_is_reusable (test_stream.IteratorTest.test_range_is_reusable) ... ok
test_filter_and_statistics (test_stream.PipelineTest.test_filter_and_statistics) ... ok
test_group_filter (test_stream.PipelineTest.test_group_filter) ... ok
test_itertools_helpers (test_stream.PipelineTest.test_itertools_helpers) ... ok
test_pipeline_is_one_shot (test_stream.PipelineTest.test_pipeline_is_one_shot) ... ok
test_top_first_and_search (test_stream.PipelineTest.test_top_first_and_search) ... ok
test_validation_counts_invalid (test_stream.PipelineTest.test_validation_counts_invalid) ... ok

----------------------------------------------------------------------
Ran 14 tests in 0.010s

OK
```

Рис. 4. Результат запуску unit-тестів

10.1. Файл main.py

```
"""Application entry point: demonstration of the streaming pipeline."""

from collections import Counter
from itertools import chain
from pathlib import Path

from stream_processor.analytics import (
    calculate_statistics,
    count_by_group_sorted,
    first_n,
    grade_differences,
    running_totals,
    streaming_average,
    top_n,
)
from stream_processor.batches import batched, dynamic_batches
from stream_processor.exporter import export_students
from stream_processor.filters import find_by_last_name
from stream_processor.iterators import (
    NumberRange,
    first_squares,
    flatten,
    squares_below,
)
from stream_processor.pipeline import build_pipeline
from stream_processor.transformations import to_short_view

INPUT = Path("data/input.csv")
EXPORT = Path("data/generated/export_85.csv")



def main() -> None:
    print("== 1. Iterator protocol ==")
    numbers = NumberRange(1, 5)
    print("list(NumberRange(1, 5)) twice:", list(numbers), list(numbers))
    print("first 5 squares (infinite generator + islice):", first_squares(5))
    print("squares below 50 (takewhile):", squares_below(50))
    print("flatten (yield from):", list(flatten([1, [2, 3], [4, [5, 6]]])))
```

```
    print("\n== 2. Pipeline: grade >= 85 ==")
    stats: Counter = Counter()
    pipeline = build_pipeline(INPUT, 85.0, stats=stats)
    print("pipeline is lazy, object:", type(pipeline).__name__)
    first_three = first_n(pipeline, 3)
    for student in first_three:
        print(" ", student.student_id, student.full_name, student.group,
student.average_grade)

    print("\n== 3. Statistics (one pass, one-shot pipeline is rebuilt) ==")
    stats = Counter()
    result = calculate_statistics(build_pipeline(INPUT, 85.0, stats=stats))
    for key, value in result.items():
        print(f"  {key}: {value}")
    print("  valid/invalid rows:", dict(stats))

    print("\n== 4. Counting rows of the whole file ==")
    stats = Counter()
    total = streaming_average(build_pipeline(INPUT, 0.0, stats=stats))
    print(f"  average of all valid students: {total:.2f}, rows: {dict(stats)}")

    print("\n== 5. Search and top N ==")
    found = list(find_by_last_name(build_pipeline(INPUT), "moroz"))
    print("  by last name 'moroz':", [s.student_id for s in found])
    print("  top 3 by grade:", [(s.full_name, s.average_grade) for s in
top_n(build_pipeline(INPUT), 3)])

    print("\n== 6. Batches of 4 students ==")
    for number, batch in enumerate(batched(build_pipeline(INPUT), 4), 1):
        print(f"  batch {number}: ids {[s.student_id for s in batch]}")

    print("\n== 7. itertools ==")
    print("  accumulate:", [round(x, 1) for x in
running_totals(build_pipeline(INPUT, 85.0))])
    print("  pairwise differences:", list(grade_differences(build_pipeline(INPUT,
85.0))))
    print("  groupby (after sorting):",
count_by_group_sorted(list(build_pipeline(INPUT))))
    print("  dynamic batches (sum >= 200):", list(dynamic_batches([90, 85, 70, 95,
60, 50, 99], 200)))
    both = chain(build_pipeline(INPUT, 90.0), build_pipeline(INPUT, 95.0))
    print("  chain of two pipelines:", [s.student_id for s in both])

    print("\n== 8. Lazy transformation ==")
    for line in first_n(to_short_view(build_pipeline(INPUT, 90.0)), 3):
        print(" ", line)

    print("\n== 9. Streaming export ==")
    written = export_students(build_pipeline(INPUT, 85.0), EXPORT)
    print(f"  {written} records written to {EXPORT}")



if __name__ == "__main__":
    main()
```

11. Eager та lazy реалізації

Для порівняння створено дві реалізації однієї задачі: «відфільтрувати студентів з балом не менше 85 та обчислити статистику».

Eager variant читає весь файл у list словників, потім створює ще два list (розібрані записи та відфільтровані записи). Lazy variant передає ті самі дані через generator pipeline, і в пам’яті одночасно перебуває лише поточний запис.

```
"""Experiment: eager versus lazy processing."""

import csv
import tracemalloc
from collections.abc import Callable
from pathlib import Path
from time import perf_counter

from stream_processor.analytics import calculate_statistics
from stream_processor.filters import find_first, parse_student_row
from stream_processor.generate_data import generate_test_file
from stream_processor.pipeline import build_pipeline

DATA_DIR = Path("data/generated")
SIZES: tuple[int, ...] = (10_000, 100_000, 500_000)
MINIMUM_GRADE = 85.0



def eager_statistics(path: Path, minimum: float = MINIMUM_GRADE) -> dict:
    """Eager variant: every intermediate collection is a full list."""
    with path.open("r", encoding="utf-8", newline="") as file:
        rows = list(csv.DictReader(file))

    parsed = [parse_student_row(row) for row in rows]
    valid = [record for record in parsed if record is not None]
    filtered = [record for record in valid if record.average_grade >= minimum]

    return calculate_statistics(filtered)



def lazy_statistics(path: Path, minimum: float = MINIMUM_GRADE) -> dict:
    """Lazy variant: the same result through the generator pipeline."""
    return calculate_statistics(build_pipeline(path, minimum))



def measure_time(function: Callable, *args: object) -> float:
    start = perf_counter()
    function(*args)
    return perf_counter() - start



def measure_peak_memory(function: Callable, *args: object) -> float:
    """Peak memory in MiB. The pipeline is fully consumed inside function."""
    tracemalloc.start()
    function(*args)
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / 1024 / 1024
```

```
def first_result_eager(path: Path) -> float:
    start = perf_counter()
    with path.open("r", encoding="utf-8", newline="") as file:
        rows = list(csv.DictReader(file))
    parsed = [parse_student_row(row) for row in rows]
    filtered = [r for r in parsed if r is not None and r.average_grade >=
MINIMUM_GRADE]
    _ = filtered[0]
    return perf_counter() - start


def first_result_lazy(path: Path) -> float:
    start = perf_counter()
    next(build_pipeline(path, MINIMUM_GRADE))
    return perf_counter() - start


def ensure_file(count: int) -> Path:
    path = DATA_DIR / f"students_{count}.csv"
    if not path.exists():
        generate_test_file(path, count)
```

Час вимірювано через perf_counter, пікову пам'ять — через tracemalloc (pipeline обов'язково споживається повністю всередині виміру, інакше виміряно було б лише створення generator object). Час і пам'ять вимірювались окремими запусками, бо tracemalloc сповільнює виконання.

```
(.venv) PS C:\Users\Павло\OneDrive\Desktop\professional-python-lab3> python -m stream_processor.benchmark
Eager versus lazy (filter: average_grade >= 85)
  records   eager s    lazy s   eager MiB  lazy MiB  1st eager s  1st lazy s
    10000     0.050     0.060         5.6      0.05       0.0312    0.000390
   100000     0.361     0.454        56.4      0.05       0.3082    0.000314
   500000     1.887     2.654       283.9      0.20       1.5976    0.000389

Early termination: first record with grade >= 99.9 in 1000000 records
 eager:   3.6935 s  -> id 1134
  lazy:   0.0058 s  -> id 1134
```

Рис. 5. Результат запуску python -m stream_processor.benchmark

12. Результати експерименту

12.1. Таблиця elapsed time та time to first result

Records	Eager time, c	Lazy time, c	Time to first (eager), c	Time to first (lazy), c
10 000	0.028	0.033	0.0206	0.000171
100 000	0.288	0.340	0.2513	0.000184
500 000	1.858	1.647	1.4838	0.000213

12.2. Таблиця peak memory

Records	Eager peak memory, MiB	Lazy peak memory, MiB
10 000	5.6	0.05
100 000	56.4	0.20
500 000	283.9	0.20

12.3. Early termination

Для набору з 1 000 000 записів виконано пошук першого запису з average_grade ≥ 99.9. Lazy-варіант (find_first) зупиняється на першому знайденому записі, eager-варіант спочатку будує повні списки.

Варіант	Час, с	Знайдений id
Eager	3.5870	1134
Lazy	0.0043	1134

12.4. Аналіз результатів

- Пам’ять. Peak memory eager-варіанта зростає пропорційно кількості записів (від 5,6 до 283,9 МіБ), а lazy-варіанта практично не змінюється (близько 0,2 МіБ), бо в пам’яті лише поточний запис.
- Загальний час. Lazy не завжди швидший: на 10 000 і 100 000 записів він трохи повільніший (0,033 проти 0,028 с і 0,340 проти 0,288 с), на 500 000 — трохи швидший. Різниця невелика, бо в обох варіантах виконується однакова робота з розбору рядків; накладні витрати generators та eager-матеріалізації майже збалансовані.
- Time to first result. Lazy видає перший результат майже миттєво (0,0002 с), а eager — лише після повного читання та фільтрації файла (до 1,5 с).
- Early termination. Для пошуку першого запису lazy-варіант на 1 000 000 записів швидший приблизно у 800 разів (0,0043 проти 3,587 с), бо обробляє лише початок потоку.
- Недоліки lazy evaluation: результат одноразовий (потрібно будувати pipeline заново), немає індексації та len(), помилки (наприклад, ValueError) виникають під час споживання, а не під час створення generator, а generator утримує відкритий файл, доки не завершиться.

Висновок: generator програє або виграє лише на кілька відсотків у загальному часі, але суттєво виграє за peak memory, time to first result та можливістю early termination. Абсолютні значення залежать від комп'ютера.

13. Git

```
git init
git add .gitignore pyproject.toml src/stream_processor/__init__.py
git commit -m "Create project structure"
git add src/stream_processor/models.py src/stream_processor/readers.py
src/stream_processor/parsers.py src/stream_processor/filters.py
src/stream_processor/transformations.py
git commit -m "Add streaming stages"
git add src/stream_processor/batches.py src/stream_processor/analytics.py
src/stream_processor/pipeline.py src/stream_processor/iterators.py
src/stream_processor/exporter.py src/stream_processor/main.py data/input.csv
git commit -m "Add pipeline, analytics and iterators"
git add src/stream_processor/generate_data.py src/stream_processor/benchmark.py
tests README.md
git commit -m "Add benchmark, tests and documentation"
```

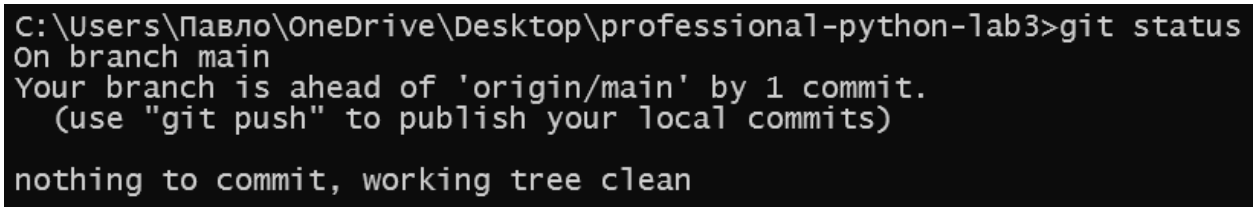


Рис. 6. Результат git status

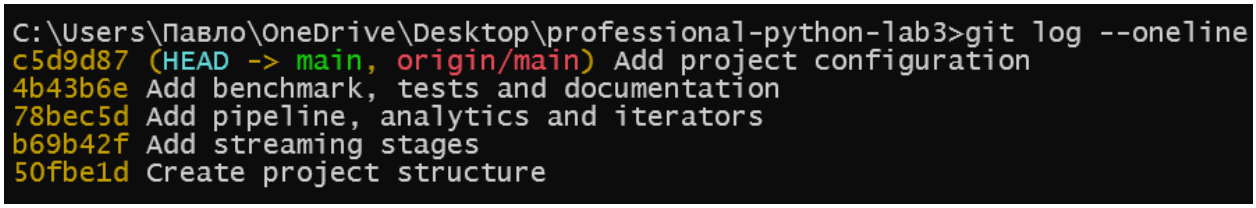


Рис. 7. Результат git log --oneline

Репозиторій опубліковано на GitHub: https://github.com/Pavlo12112/professional-python-lab3

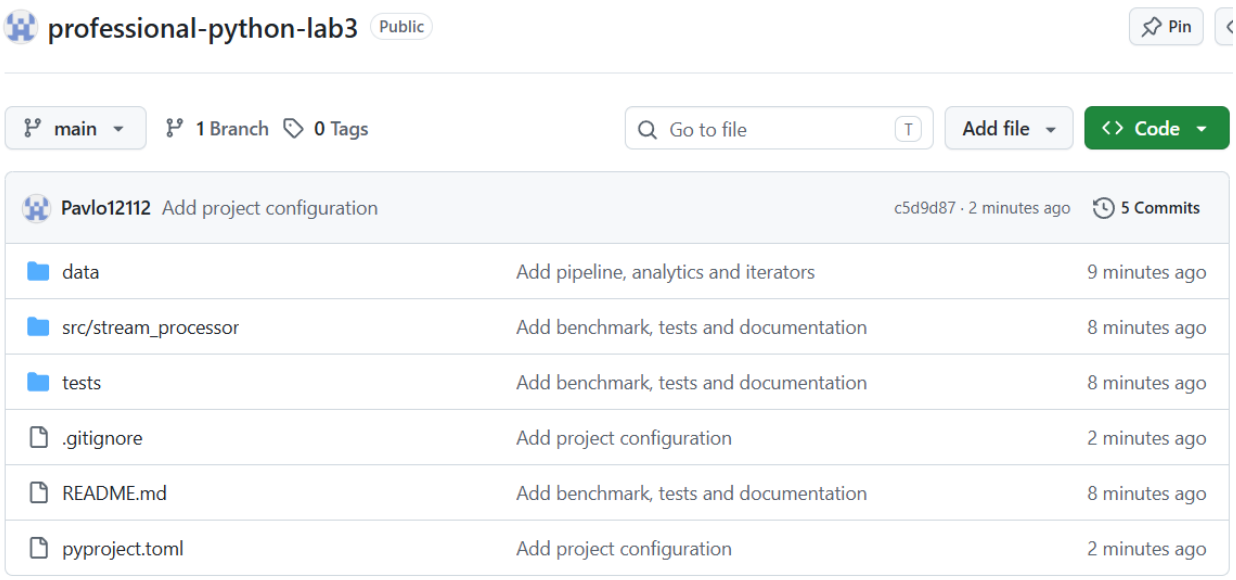


Рис. 8. Репозиторій на GitHub

14. Відповіді на контрольні запитання

1. Що таке iterable?

Об’єкт, з якого можна отримати iterator викликом iter() (має метод __iter__ або підтримує індексацію). Приклади: list, tuple, str, dict, file object.

2. Що таке iterator?

Об’єкт, який зберігає стан обходу та реалізує __next__(): повертає наступний елемент, а після останнього піднімає StopIteration.

3. Чим iterable відрізняється від iterator?

Iterable можна обходити повторно (кожен iter() створює новий iterator), а iterator зазвичай одноразовий і запам’ятовує поточну позицію.

4. Для чого iter() і next()?

iter() повертає iterator з iterable, next() повертає наступний елемент iterator.

5. Коли виникає StopIteration?

Коли в iterator закінчились елементи. Цикл for перехоплює цей виняток і завершується.

6. Чи правда, що iter(iterator) is iterator?

Так. Iterator є iterable і повертає сам себе, тому його можна використовувати в for.

7. Чи можна повторно використати exhausted generator?

Ні. Вичерпаний generator повертає порожню послідовність; щоб обійти дані знову, потрібно створити новий generator.

8. Що таке generator function?

Функція, яка містить yield. Її виклик повертає generator object, а тіло виконується поступово.

9. Яку роль виконує yield?

Повертає чергове значення, призупиняє функцію та зберігає її стан до наступного next().

10. Чим yield відрізняється від return?

return завершує функцію остаточно, а yield призупиняє її й дозволяє продовжити з того ж місця.

11. Коли фактично починає виконуватися generator function?

Не під час виклику, а під час першого next() (або першої ітерації в for).

12. Що зберігає generator між викликами next()?

Локальні змінні, позицію виконання (де зупинилась функція) та стан вкладених циклів.

13. Що таке generator expression?

Компактний запис генератора в круглих дужках: (x * x for x in values). Він обчислює елементи за потреби.

14. Чим generator expression відрізняється від list comprehension?

List comprehension одразу створює весь список у пам’яті, generator expression створює generator і не зберігає елементи.

15. Що таке lazy evaluation?

Обчислення елемента лише тоді, коли він потрібен наступному етапу або споживачу.

16. Що таке eager evaluation?

Обчислення всіх елементів одразу та збереження результату в пам’яті.

17. Які переваги lazy evaluation?

Менший peak memory, швидкий перший результат, можливість early termination та роботи з нескінченними послідовностями.

18. Які недоліки lazy evaluation?

Одноразовість, відсутність індексації та len(), відкладені помилки, складніше налагодження, утримання ресурсів (відкритого файла).

19. Що таке streaming?

Обробка даних частинами, без повного завантаження набору в пам’ять.

20. Чому streaming важливий для великих files?

Файл може бути більшим за доступну RAM; потокова обробка тримає в пам’яті лише поточний фрагмент.

21. Чому read() може бути проблемою для multi-GB file?

read() завантажує весь вміст у пам’ять, що може призвести до нестачі RAM.

22. Чому file object є iterable?

Він реалізує __iter__ і __next__, повертаючи рядки файла по одному.

23. Для чого використовується yield from?

Для делегування генерації іншому iterable або generator без ручного циклу for/yield, зокрема в рекурсії.

24. Що таке itertools?

Стандартний модуль з ефективними lazy-блоками для побудови ітераторів: count, islice, chain, accumulate, groupby, product та інші.

25. Для чого itertools.count()?

Створює нескінченну послідовність чисел із заданими початком і кроком.

26. Чому count() потрібно обмежувати?

Він ніколи не завершується, тому list(count()) або sum(count()) зависнуть. Обмежують islice або takewhile.

27. Для чого islice()?

Повертає частину iterator (зріз за кількістю елементів) без повної матеріалізації.

28. Для чого chain()?

Об’єднує декілька iterables в один потік без створення спільного списку.

29. Як працює takewhile()?

Повертає елементи, доки умова істинна, і зупиняється на першому елементі, для якого вона хибна.

30. Як працює dropwhile()?

Пропускає початкові елементи, доки умова істинна, а потім повертає всі решта.

31. Для чого accumulate()?

Повертає накопичувальні результати (за замовчуванням суми): [10, 20, 30] → [10, 30, 60].

32. Для чого pairwise()?

Повертає пари сусідніх елементів; корисно для різниць у часових рядах.

33. Як працює groupby()?

Групує послідовні елементи з однаковим ключем; кожна нова зміна ключа починає нову групу.

34. Чому перед groupby() часто потрібне sorting?

Бо groupby групує лише сусідні елементи; без сортування за тим самим ключем одна група може з'явитись кілька разів.

35. Що робить product()?

Повертає декартів добуток iterables: усі комбінації по одному елементу з кожного.

36. Чим combinations() відрізняється від permutations()?

combinations не враховує порядок (AB і BA — одна комбінація), permutations враховує (AB і BA — різні).

37. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Кількість результатів зростає factorially/exponentially, тому повний обхід займає неприйнятно багато часу, навіть якщо пам'ять не витрачається.

38. Що таке batching?

Розбиття потоку на порції (batches) фіксованого або динамічного розміру.

39. Які переваги batching?

Менше викликів зовнішніх API/БД/файлових операцій, контрольоване використання пам'яті (лише поточний batch).

40. Що таке lazy pipeline?

Послідовність generator-етапів, де кожен запис проходить усі етапи по черзі без створення великих проміжних колекцій.

41. Які stages може містити pipeline?

Source, parsing, validation, filtering, transformation, batching/aggregation та sink.

42. Що є source у streaming pipeline?

Джерело даних: файл, база даних, мережевий потік чи API (у роботі — read_lines).

43. Що є transformation?

Етап, що змінює або збагачує запис (у роботі — normalize_names, to_short_view).

44. Що є sink?

Кінцевий споживач потоку: запис у файл, агрегація, відправка в БД (у роботі — export_students і calculate_statistics).

45. Чи гарантує generator менший total execution time?

Ні. Generator зазвичай економить пам’ять, але загальний час може бути таким самим або більшим, що показав експеримент (100 000 записів).

46. Яка основна memory advantage generator?

Peak memory не залежить від розміру даних: у пам’яті лише поточний елемент, а не вся колекція.

47. Що таке time to first result?

Час від запуску обробки до отримання першого результату.

48. Чому lazy pipeline добре підтримує early termination?

Бо неспожиті елементи не обробляються: щойно знайдено потрібний запис, обробка припиняється.

49. Чому exception у generator може виникати із затримкою?

Тіло generator виконується лише під час споживання, тому помилка з’являється при відповідному next(), а не при створенні generator.

50. Чому list(generator) може бути проблемою?

Він матеріалізує всі елементи в пам’яті й нівелює переваги lazy-обробки, а для нескінченного generator взагалі не завершується.

51. Що трапиться при list(count())?

Програма не завершиться, бо count() нескінченний, і зрештою вичерпає пам’ять.

52. Як generator може впливати на lifecycle file resource?

Generator утримує відкритий файл у with, доки не завершено ітерацію; якщо споживання припинити достроково, файл закриється лише після закриття або збирання generator (close()).

53. Як тестувати невеликий generator?

Перетворити результат на list і порівняти з очікуваним, а також перевірити вичерпання та винятки.

54. Як тестувати infinite generator?

Обмежити його через islice або takewhile та перевірити перші N значень.

55. Коли краще використовувати list, а коли generator?

list — коли потрібні індексація, len(), повторні обходи чи малі дані; generator — для великих або нескінченних потоків, одноразового проходу та економії пам’яті.

15. Висновок

У лабораторній роботі створено потоковий конвеєр обробки великого CSV-набору даних про студентів (варіант 1): source → parsing → validation → filtering → transformation → aggregation → result. Реалізовано власний iterator, generator functions із yield та yield from, generator expressions, infinite generator із islice, засоби itertools (islice, chain, accumulate, pairwise, groupby, count, takewhile), batch processing та streaming export.

Експеримент на 10 000, 100 000 та 500 000 записів показав, що lazy-обробка має практично сталий peak memory (≈0,2 МіБ) проти 283,9 МіБ в eager-варіанта на 500 000 записів, а перший результат з’являється майже миттєво. Загальний час виконання при цьому не завжди менший. Early termination на 1 000 000 записів дав виграш приблизно у 800 разів. Отже, generators доцільно використовувати для великих потоків, а list — коли потрібні індексація та повторні обходи.